Full Benchmark List Used in Our Experimental Evaluation

This document provides the complete list of benchmarks used in our evaluation. For each benchmark, we report its name, threading mode (ST for Single-Threaded, MT for Multi-Threaded), and a brief description.

Table 1: List of Java benchmarks from the DaCapo suite used in our evaluation.

Benchmark	**ST/MT**	**Description**
`fop`	ST	Parses XSL-FO documents and renders them into PDF output.
`luindex`	ST	Uses Apache Lucene [1] to build an index over a document corpus.
`lusearch`	MT	Uses Apache Lucene [1] to run keyword searches over an indexed document corpus.
`pmd`	MT	Uses PMD [35] to statically analyze Java source code and report common programming issues.
`sunflow`	MT	Uses the Sunflow rendering engine [24] to ray-trace a 3D scene and produce rendered images.
`xalan`	MT	Uses Xalan [2] to apply XSLT.

Table 2: List of Scala benchmarks from the DaCapo con Scala suite used in our evaluation.

Benchmark	**ST/MT**	**Description**
`apparat`	ST/MT	Uses Apparat [7] to analyze JVM classfiles and apply bytecode transformations.
`factorie`	ST	Uses Factorie toolkit [27] to run probabilistic modeling and inference over a machine-learning workload.
`kiama`	ST	Uses Kiama [36] to process programs with rewriting and attribute-grammar evaluation.
`scalac`	ST	Uses the Scalac compiler [42] to compile the Scalap classfile decoder [46].
`scaladoc`	ST	Uses Scaladoc [45] to generate API documentation for the Scalap classfile decoder [46].
`scalap`	ST	Uses Scalap classfile decoder [46] to decode Scala/JVM classfiles and inspect signatures and metadata.
`scalariform`	ST	Uses Scalariform [45] to parse and format Scala source code according to style rules.
`scalaxb`	ST	Uses Scalaxb [12] to compile W3C XML Schemas [32] into Scala bindings.
`tmt`	ST/MT	Uses TMT [8] to run a topic-modeling and text-mining pipeline over a text corpus.

Table 3: List of Scala benchmarks from the Renaissance suite used in our evaluation.

Benchmark	ST/MT	Description
`akka-uct`	MT	Runs the Unbalanced Cobwebbed Tree actor workload using the Akka framework [33, 37].
`scala-kmeans`	ST	Runs the k-means clustering algorithm [23] using Scala collections.
`fj-kmeans`	MT	Runs the k-means clustering algorithm using the Fork/Join framework [31].
`mnemonics`	ST	Solves the phone-number mnemonics problem using JDK streams.
`par-mnemonics`	MT	Solves the phone-number mnemonics problem using parallel streams.
`philosophers`	MT	Simulates a variant of the dining philosophers problem [25] using ScalaSTM [4].
`scrabble`	MT	Solves the Scrabble puzzle using Java streams.
`rx-scrabble`	MT	Solves the Scrabble puzzle using RxJava and reactive-stream operators.
`finagle-http`	MT	Sends many small HTTP requests using Twitter Finagle [41] and awaits responses from a Finagle HTTP server.
`future-genetic`	MT	Runs a genetic algorithm using the Jenetics library [43] and futures for parallelism.
`reactors`	MT	Runs a sequence of actor-style workloads on Reactors.IO framework [5], inspired by Savina microbenchmarks [18].
`scala-doku`	ST	Solves Sudoku puzzles using Scala collections and backtracking search.
`scala-stm-bench7`	MT	Runs the STMBench7 benchmark [15] using ScalaSTM.

Table 4: List of C/C++ benchmarks from the PolyBench/C suite used in our evaluation. All benchmarks are single-threaded.

Benchmark	Description
`correlation`	Computes a correlation matrix from an input data matrix.
`covariance`	Computes a covariance matrix from an input data matrix.
`gemm`	Computes a dense matrix multiplication and update: $C \leftarrow \alpha AB + \beta C$.
`gemver`	Performs a sequence of vector updates and a matrix–vector products (GEMVER kernel [17]).
`gesummv`	Computes two matrix–vector products and combines them: $y \leftarrow \alpha Ax + \beta Bx$.
`symm`	Multiplies a symmetric matrix by a dense matrix and accumulates the result (SYMM kernel [19]).
`syr2k`	Updates a symmetric matrix with two rank-k products (SYR2K kernel [20]).
`syrk`	Updates a symmetric matrix: $C \leftarrow \beta C + \alpha AA^T$ (SYRK kernel [21]).
`trmm`	Computes triangular matrix–matrix multiply: $B \leftarrow \alpha AB$ (TRMM kernel [22]).
`2mm`	Computes two matrix multiplications: $D \leftarrow AB$ and $E \leftarrow CD$.
`3mm`	Computes three matrix multiplications: $E \leftarrow AB$, $F \leftarrow CD$, and $G \leftarrow EF$.
`atax`	Computes $y \leftarrow A^T(Ax)$ (ATAX kernel [29]).
`bicg`	Runs the BiCG sub-kernel of the BiCGStab linear solver [39].
`doitgen`	Runs the DOITGEN multiresolution analysis kernel (MADNESS [16]).

Continued on next page

Benchmark	Description
`mvt`	Performs a matrix–vector product and a transpose.
`cholesky`	Computes the Cholesky decomposition [13] of a square matrix.
`durbin`	Solves a Toeplitz linear system [14] using Durbin's recursion [10].
`gramschmidt`	Performs Gram–Schmidt decomposition [13, 3] by orthogonalizing the columns of a matrix.
`ludcmp`	Performs LU decomposition [13, 40] using the LUDCMP solver kernel.
`lu`	Performs LU decomposition [13, 40] of a dense matrix.
`trisolv`	Solves a triangular linear system (triangular solver kernel).
`deriche`	Applies the Deriche image filter [9] kernel to a 2D image.
`floyd-warshall`	Runs the Floyd–Warshall kernel [11] to compute all-pairs shortest paths.
`nussinov`	Runs the Nussinov kernel [30] to compute RNA secondary structure using dynamic programming.
`adi`	Runs an Alternating Direction Implicit (ADI [34]) solver kernel.
`fdtd-2d`	Runs a 2D finite-difference time-domain (FDTD [44]) stencil kernel.
`heat-3d`	Runs a 3D heat-equation stencil kernel [38].
`jacobi-1d`	Runs a 1D Jacobi stencil computation.
`jacobi-2d`	Runs a 2D Jacobi stencil computation.
`seidel-2d`	Runs a 2D Seidel stencil computation.

Table 5: List of C/C++ benchmarks from the PARSEC suite used in our evaluation. All benchmarks are multi-threaded.

Benchmark	Description
`blackscholes`	Prices a portfolio of European options using the Black–Scholes model, emphasizing floating-point computation.
`bodytrack`	Tracks a markerless 3D human pose across multi-camera image sequences using an annealed particle filter with edge and silhouette features.
`facesim`	Simulates physics-based facial animation by applying time-varying muscle activations to a face model to generate realistic motion.
`ferret`	Performs content-based image similarity search using a pipelined workflow (segmentation, feature extraction, hashing-based candidate selection, and ranking).
`fluidanimate`	Simulates an incompressible fluid for interactive animation using an SPH-based method and specialized kernels for stability and speed.
`freqmine`	Mines frequent itemsets using an FP-growth approach (FP-tree based), representative of association-rule mining workloads.
`swaptions`	Prices a portfolio of swaptions under the Heath–Jarrow–Morton framework using Monte Carlo simulation [28] with randomized interest-rate paths.
`vips`	Runs a multi-stage image transformation pipeline (e.g., crop, affine transforms, convolution) using the VIPS image processing system [26, 6].

References

[1] Apache. 2010. Apache Lucene-overview. `http://lucene.apache.org/iava/docs/`

[2] Apache. 2023. xalan. `https://xalan.apache.org/`

[3] Åke Björck. 1967. Solving linear least squares problems by Gram-Schmidt orthogonalization. *BIT Numerical Mathematics* 7, 1 (1967), 1–21.

[4] the Scala STM Expert Group Bronson, N. 2023. Library-based software transactional memory for scala. https://nbronson.github.io/scala-stm/faq.html

[5] Scala community. 2017. Reactors.IO. https://github.com/reactors-io/reactors-io.github.io

[6] John Cupitt and Kirk Martinez. 1996. VIPS: an image processing system for large images. In *Very High Resolution and Quality Imaging*, Vol. 2663. SPIE, 19–28.

[7] Technische Universität Darmstadt. 2012. DaCapo con Scala apparat. https://benchmarks.scalabench.org/modules/apparat-dacapo-benchmark

[8] Technische Universität Darmstadt. 2012. DaCapo con Scala tmt. https://benchmarks.scalabench.org/modules/tmt-dacapo-benchmark

[9] Rachid Deriche. 1987. Using Canny's criteria to derive a recursively implemented optimal edge detector. *International journal of computer vision* 1, 2 (1987), 167–187.

[10] James Durbin. 1960. The fitting of time-series models. *Revue de l'Institut International de Statistique* (1960), 233–244.

[11] Robert W Floyd. 1962. Algorithm 97: shortest path. *Commun. ACM* 5, 6 (1962), 345–345.

[12] Shudi Sandy Gao, C Michael Sperberg-McQueen, and Henry Thompson. 2012. W3C XML schema definition language (XSD) 1.1 part 1: Structures. (2012).

[13] Gene H Golub and Charles F Van Loan. 2013. *Matrix computations*. JHU press.

[14] Ulf Grenander. 1958. *Toeplitz forms and their applications*. Univ of California Press.

[15] Rachid Guerraoui, Michal Kapalka, and Jan Vitek. 2007. Stmbench7: a benchmark for software transactional memory. *ACM SIGOPS Operating Systems Review* 41, 3 (2007), 315–324.

[16] Robert J Harrison, Gregory Beylkin, Florian A Bischoff, Justus A Calvin, George I Fann, Jacob Fosso-Tande, Diego Galindo, Jeff R Hammond, Rebecca Hartman-Baker, Judith C Hill, et al. 2016. MADNESS: A multiresolution, adaptive numerical environment for scientific simulation. *SIAM journal on scientific computing* 38, 5 (2016), S123–S142.

[17] Gary W Howell, James W Demmel, Charles T Fulton, Sven Hammarling, and Karen Marmol. 2008. Cache efficient bidiagonalization using BLAS 2.5 operators. *ACM Transactions on Mathematical Software (TOMS)* 34, 3 (2008), 1–33.

[18] Shams M Imam and Vivek Sarkar. 2014. Savina-an actor benchmark suite: Enabling empirical evaluation of actor libraries. In *Proceedings of the 4th International Workshop on Programming Based on Actors Agents & Decentralized Control*. 67–80.

[19] Intel. 2024. SYMM. https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2024-0/symm.html

[20] Intel. 2024. SYR2K. https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2024-0/syr2k.html

[21] Intel. 2024. SYRK. https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2024-0/syrk.html

[22] Intel. 2024. TRMM. https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2024-0/trmm.html

[23] Tapas Kanungo, David M Mount, Nathan S Netanyahu, Christine D Piatko, Ruth Silverman, and Angela Y Wu. 2002. An efficient k-means clustering algorithm: Analysis and implementation. *IEEE transactions on pattern analysis and machine intelligence* 24, 7 (2002), 881–892.

[24] C. Kulla. 2023. Sunflow render system documentation. `https://sunflow.sourceforge.net/`

[25] Daniel Lehmann and Michael O Rabin. 1981. On the advantages of free choice: A symmetric and fully distributed solution to the dining philosophers problem. In *Proceedings of the 8th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 133–138.

[26] Kirk Martinez and John Cupitt. 2005. VIPS-a highly tuned image processing software architecture. In *IEEE International Conference on Image Processing 2005*, Vol. 2. IEEE, II–574.

[27] Andrew McCallum, Karl Schultz, and Sameer Singh. 2009. FACTORIE: Probabilistic Programming via Imperatively Defined Factor Graphs. In *Neural Information Processing Systems (NIPS)*.

[28] Nicholas Metropolis and Stanislaw Ulam. 1949. The monte carlo method. *Journal of the American statistical association* 44, 247 (1949), 335–341.

[29] Microsoft. 2015. ATAX. `https://github.com/microsoft/test-suite/blob/master/SingleSource/Benchmarks/Polybench/linear-algebra/kernels/atax/atax.c`

[30] Ruth Nussinov and Ann B Jacobson. 1980. Fast algorithm for predicting the secondary structure of single-stranded RNA. *Proceedings of the National Academy of Sciences* 77, 11 (1980), 6309–6313.

[31] Oracle. 2024. Fork/Join Java framework. `https://docs.oracle.com/javase/tutorial/essential/concurrency/forkjoin.html`

[32] ScalaXB org. 2023. ScalaXb. `https://scalaxb.org/`

[33] Hector Veiga Ortiz and Piyush Mishra. 2017. *Akka Cookbook*. Packt Publishing Ltd.

[34] Donald W Peaceman and Henry H Rachford, Jr. 1955. The numerical solution of parabolic and elliptic differential equations. *Journal of the Society for industrial and Applied Mathematics* 3, 1 (1955), 28–41.

[35] PMD. 2025. pmd. `https://pmd.github.io/`

[36] Anthony Sloane. 2023. Kiama. `https://inkytonik.github.io/projects/kiama/`

[37] Satish Narayana Srirama, Freddy Marcelo Surriabre Dick, and Mainak Adhikari. 2021. Akka framework based on the actor model for executing distributed fog computing applications. *Future Generation Computer Systems* 117 (2021), 439–452.

[38] John C Strikwerda. 2004. *Finite difference schemes and partial differential equations*. SIAM.

[39] Inc. The MathWorks. 2015. BiCGStab. `https://www.mathworks.com/help/matlab/ref/bicgstab.html`

[40] Lloyd N Trefethen and David Bau. 2022. *Numerical linear algebra*. SIAM.

[41] Twiter. 2023. Twiter finagle. `https://twitter.github.io/finagle`

[42] Vincent Van der Leun. 2017. *Introduction to JVM Languages*. Packt Publishing Ltd.

[43] F. Wilhelmstötter. 2021. Jenetics. `http://jenetics.io`

[44] Kane Yee. 1966. Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media. *IEEE Transactions on antennas and propagation* 14, 3 (1966), 302–307.

[45] Switzerland École Polytechnique Fédérale Lausanne (EPFL) Lausanne. 2023. Scala documentation. https://www.scala-lang.org/

[46] Switzerland École Polytechnique Fédérale Lausanne (EPFL) Lausanne. 2024. Scalap documentation. https://javadoc.io/doc/org.scala-lang/scalap/latest/index.html